# [Day6] 5. Hierarchical 패턴과 확장 (Extra) (1)

| | |
|---|---|
| Category | |
| Date | 2026년 8월 19일 오후 2:39 |
| Published | ☐ |
| module | Day 6 |
| order | 6 |
| 강의실(classroom) | sds-ax |

5-1. Hierarchical 패턴 (실습)

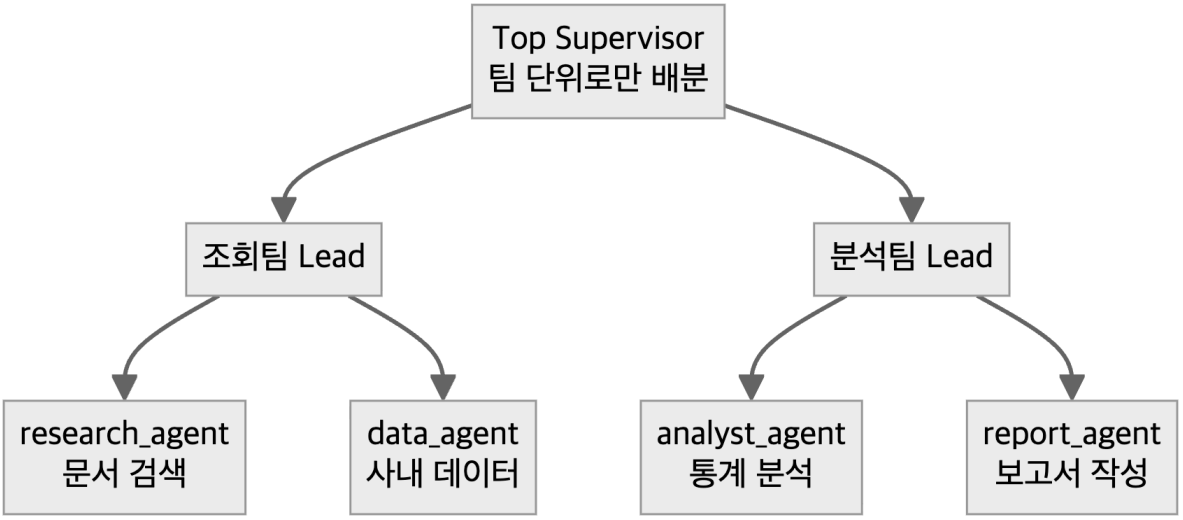
5-2. 추가로 해볼 만한 것

## 5-1. Hierarchical 패턴 (실습)

> 💡 Supervisor 밑에 Supervisor를 두는 계층 구조를 직접 돌려봅니다.

오늘 만든 Supervisor는 서브 Agent가 셋입니다. 그런데 시스템이 커져 6개, 8개로 늘어나면 어떻게 될까요? Supervisor 하나가 전부를 알아야 하는 구조가 다시 복잡해집니다. 1번째 세션에서 본 단일 Agent의 문제가 이번에는 Supervisor에서 되풀이되는 것입니다.

풀이도 같습니다. 또 나눕니다. 이번에는 Agent가 아니라 Supervisor를 나눕니다. 이것이 Hierarchical(계층형) 패턴입니다. 팀 안에 팀을 만드는 구조로, 조회팀과 분석팀을 각각의 Lead(하위 Supervisor)가 이끕니다. 상위 관리자는 팀 단위로만 배분하고 팀 안에서 누가 일할지는 Lead가 정합니다.




그러면 1번째 세션에서 3번째 표였던 Network는 어떻게 다를까요? 차이는 handoff(다른 Agent에게 작업을 넘기는 것)의 방향에 있습니다.

- top-down handoff: 위에서 아래로만 일을 넘깁니다. Supervisor와 Hierarchical이 이 방식입니다. `create_supervisor`가 `transfer_to_이름` 도구를 자동으로 만들어 주므로 별도 코드가 필요 없습니다.
- peer-to-peer handoff: 위아래 없이 동등한 Agent끼리 직접 넘깁니다. Network형이 이 방식입니다. `Command(goto=대상, graph=Command.PARENT)`를 반환하는 handoff 도구를 직접 만들어 각 Agent에 넘겨야 합니다.

두 방식 모두 안에서는 같은 `Command(goto=...)`가 동작합니다. 차이는 누가 그 도구를 들고 있느냐입니다. 오늘은 위에서 아래로만 넘기는 쪽을 한 번 더 포개 계층을 만들어봅니다.

**따라하기**

1. 팀 그래프를 만들어 상위 Supervisor의 서브 Agent 자리에 그대로 넣습니다. 새 API 없이 `create_supervisor`를 두 번 포개어 쓰면 됩니다. 아래 코드를 `hierarchical_demo.py`로 저장해 실행합니다.

```
# hierarchical_demo.py - 조회팀과 분석팀을 지휘하는 계층 구조
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from langchain.agents import create_agent
from langgraph_supervisor import create_supervisor

load_dotenv()
```

```
MODEL = "us.anthropic.claude-sonnet-4-5-20250929-v1:0"

def llm():
    return ChatBedrockConverse(model=MODEL, region_name
="us-east-1", temperature=0)

# 교육용 더미 도구입니다
@tool
def retrieve_docs(query: str) -> str:
    """사내 규정 문서를 검색한다."""
    return f"[문서] {query}: 장비 사외 반출 시 팀장 승인 필
수"

@tool
def search_db(target: str) -> str:
    """사내 데이터를 조회한다."""
    return f"[데이터] {target}: 노트북 2대, GPU 서버 1대"

@tool
def analyze(data: str) -> str:
    """데이터를 통계 분석한다."""
    return f"[분석] {data} 결과: 자산 집중도 높음, 분산 배치
권장"

@tool
def write_report(content: str) -> str:
    """마크다운 보고서를 작성한다."""
    return f"# 보고서\n\n{content}"

# 1) 조회팀: 문서 검색과 데이터 조회를 조회팀 Lead가 지휘
research_agent = create_agent(model=llm(), tools=[retrie
ve_docs],
                              system_prompt="문서
검색 전담.", name="research_agent")
data_agent = create_agent(model=llm(), tools=[search_d
b],
                          system_prompt="사내 데이
터 조회 전담.", name="data_agent")
```

```
research_team = create_supervisor(
    [research_agent, data_agent],
    model=llm(),
    prompt="너는 조회팀 Lead다. 문서 검색과 데이터 조회 작업을 분배하라.",
).compile(name="research_team")   # 팀 그래프는 compile 시 name이 필수입니다

# 2) 분석팀: 분석과 보고서를 분석팀 Lead가 지휘
analyst_agent = create_agent(model=llm(), tools=[analyze],
                             system_prompt="통계 분석 전담.", name="analyst_agent")
report_agent = create_agent(model=llm(), tools=[write_report],
                            system_prompt="보고서 작성 전담.", name="report_agent")

analysis_team = create_supervisor(
    [analyst_agent, report_agent],
    model=llm(),
    prompt="너는 분석팀 Lead다. 분석과 보고서 작업을 분배하라.",
).compile(name="analysis_team")

# 3) 최상위: 두 팀을 상위 관리자가 지휘 (팀 그래프를 Agent처럼 넣습니다)
top = create_supervisor(
    [research_team, analysis_team],
    model=llm(),
    prompt=(
        "너는 회사 전체 Supervisor다.\n"
        "- 데이터, 문서 조회는 research_team\n"
        "- 분석, 보고서 작성은 analysis_team\n"
        "조회가 먼저 필요하면 research_team부터 호출하라."
    ),
).compile()
```

```
if __name__ == "__main__":
    q = "클라우드운영팀 GPU 자산을 조회해서 분석하고 보고서로 만들어줘"
    print(f"질문: {q}\n")
    for event in top.stream(
        {"messages": [HumanMessage(content=q)]},
        stream_mode="updates",
        subgraphs=True,        # 팀 내부 흐름까지 추적합니다
        config={"recursion_limit": 40},
    ):
        path, update = event   # subgraphs=True면 경로와 업데이트 튜플로 옵니다
        for node in update:
            print(f"  {'/'.join(map(str, path)) or 'top'} -> {node}")
```

2. 출력에서 Top Supervisor가 팀 단위로만 배분하고 팀 안에서 어느 담당을 쓸지는 각 팀 Lead가 결정하는 흐름을 확인합니다.

**막힐 때**

- `compile(name=...)`을 빠뜨렸습니다: 팀 그래프를 상위에 넣으려면 compile 시 name이 필수입니다. 이 name이 상위 Supervisor의 handoff 도구 이름이 됩니다.
- `subgraphs=True` 출력 형식이 다릅니다: 이 옵션을 켜면 이벤트가 경로와 업데이트 튜플로 바뀝니다. 기존 스트리밍 처리 코드를 그대로 쓰면 값 풀기 오류가 납니다.

**확인**: research_team이 먼저 호출되고 그 안에서 data_agent가 일한 뒤 analysis_team으로 넘어가는 순서가 출력되면 성공입니다.

Hierarchical로 갈지는 서브 Agent가 5개를 넘고 자연스러운 팀 경계가 보일 때 검토합니다. 그 전에는 평평한 Supervisor가 낫습니다. 계층이 하나 늘 때마다 모델 호출과 지연도 늘어나기 때문입니다.

자주 빠지는 함정은 다음과 같습니다.

- 도구마다 Agent를 만들어 너무 잘게 나눔
- 특정 Agent에만 일이 몰림
- Supervisor 프롬프트가 거대해짐
- 순환 루프. `recursion_limit`과 명시적 종료 조건으로 막습니다.

💡 팀 안에 팀을 두는 계층 구조도 같은 API로 조립된다는 걸 확인했습니다.

## 5-2. 추가로 해볼 만한 것

▼ API 서버로 노출하기

완성한 Supervisor 앱도 LangGraph 그래프이므로 보통의 그래프와 같은 방법으로 API 서버로 낼 수 있습니다. 가장 단순한 방법은 FastAPI로 감싸는 것입니다. 아래 코드를 `api_server.py`로 저장해 실행합니다. 이 코드는 `final_scenario`에서 `build_app`과 `get_text`를 가져오므로 4번째 세션에서 만든 `final_scenario.py`와 같은 폴더에서 실행해야 합니다.

```
# api_server.py - 종합 시나리오 앱을 API로 노출
# 이 실습에만 필요한 패키지입니다: pip install fastapi uvicor
n
from contextlib import asynccontextmanager
from fastapi import FastAPI
from pydantic import BaseModel
from langchain_core.messages import HumanMessage
from final_scenario import build_app, get_text

graph_app = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    global graph_app
    graph_app = await build_app()
    yield

api = FastAPI(lifespan=lifespan)

class Query(BaseModel):
    question: str

@api.post("/chat")
async def chat(q: Query):
    result = await graph_app.ainvoke({"messages": [Human
Message(q.question)]})
```

```
    return {"answer": get_text(result["messages"][-1])}

# 실행: uvicorn api_server:api --port 8000
# 테스트: Invoke-RestMethod -Uri http://localhost:8000/ch
at -Method Post -ContentType "application/json" -Body
'{"question": "출장 식비 한도 얼마야?"}'
# 응답은 PowerShell이 자동으로 객체로 보여 줍니다. 원문이 보고
싶으면 위 명령 뒤에 | ConvertTo-Json 을 붙입니다
```

`langgraph.json`에 그래프를 등록하고 `langgraph dev`를 실행하면 스트리밍과 스레드 관리, Studio UI까지 포함된 개발 서버가 뜹니다. 배포까지 고려하면 이 경로가 표준이지만 오늘은 이런 것이 있다 정도만 알아 두면 충분할 것 같습니다!